



Universidad
Indoamérica

UNITÉ 8 · FONDEMENTS DES SYSTÈMES D'EXPLOITATION

Réseau: les sockets

Universidad Tecnológica Indoamérica

Laboratoire TD7

Deux modes de connexion

Connecté · SOCK_STREAM (TCP)

- La connexion s'établit une fois; ensuite, on échange les données
- Comme un **téléphone**
- Un **flux d'octets** fiable, comme un tube: détecte les erreurs, retransmet, garde l'ordre

Non connecté · SOCK_DGRAM (UDP)

- Chaque envoi est un message indépendant (**datagramme**)
- Comme un **télégramme**
- Aucune garantie: pertes, duplications, désordre

Flux contre datagrammes, mesuré

Sortie réelle

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3 "two", recvfrom #3: 5
"three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8 received, 9992 lost
kernel counter Udp RcvbufErrors grew by 9992
```

- TCP ne conserve pas les frontières: c'est **le protocole applicatif** qui marque où finit chaque message (une ligne, une longueur...)
- UDP perd des datagrammes **sans prévenir personne**, même sur la même machine

```
int socket(int domain, int type, int protocol);
```

domain	AF_UNIX : local, l'adresse est un chemin · AF_INET : IPv4 · AF_INET6 : IPv6
type	SOCK_STREAM : connecté · SOCK_DGRAM : non connecté · SOCK_RAW : paquets bruts (exige CAP_NET_RAW)
+ options	SOCK_NONBLOCK , SOCK_CLOEXEC (Linux, depuis 2.6.27): combinés avec dans type
protocol	0 : le système choisit · IPPROTO_TCP · IPPROTO_UDP

Correction (diapositive 182): « AF_UNIX: local (127.0.0.1) » mélange deux choses. 127.0.0.1, c'est AF_INET sur l'interface de bouclage; AF_UNIX n'utilise pas IP.

Domaine local: un fichier de type socket

Sortie réelle

```
$ ./local fso.sock  
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path holds 108 bytes  
the server asks the kernel who connected: pid = mine, uid 1000 (my user)  
read 5 bytes; after close the file stays until unlink  
still there after close: yes
```

- `bind` crée une entrée dans le système de fichiers (le `s` de `ls -l`); elle reste jusqu'à `unlink`
- Le noyau indique au serveur **qui** s'est connecté (`SO_PEERCRED`): c'est ainsi que `systemd`, `D-Bus` ou `/var/run/docker.sock` décident des permissions

Domaine Internet: adresse + port

- Un port (16 bits) identifie le service dans la machine
- Ports < 1024: réservés aux privilégiés (`CAP_NET_BIND_SERVICE`); sous Linux, la limite est `net.ipv4.ip_unprivileged_port_start`
- Noms des services: `/etc/services`

Sortie réelle

```
$ ./addr 7000
...
ip_unprivileged_port_start = 1024
sin_port = htons(80):      bind: Permission denied
...
```

SERVICE	PORT	AUJOURD'HUI
ftp	21/tcp	non chiffré; on utilise sftp ou https
telnet	23/tcp	non chiffré; remplacé par ssh (22/tcp)
http	80/tcp	redirige presque toujours vers https
https	443/tcp, 443/udp	TLS; HTTP/3 fonctionne sur UDP (QUIC)

bind : adresse + port

C

```
int bind(int sock, const struct sockaddr *addr, socklen_t len)
struct sockaddr { sa_family_t sa_family; char sa_data[14]; }
struct sockaddr_un {          /* AF_UNIX (= AF_LOCAL) */
    sa_family_t sun_family;
    char        sun_path[108]; /* a path */
};
struct sockaddr_in {          /* AF_INET */
    sa_family_t  sin_family;
    in_port_t    sin_port;    /* network order */
    struct in_addr sin_addr;   /* 4 bytes */
    unsigned char sin_zero[8];
};
/* AF_INET6: struct sockaddr_in6 (16-byte address) */
```

- Chaque domaine a sa propre structure; toutes se passent comme `struct sockaddr *` avec leur taille
- `struct sockaddr_storage` peut contenir n'importe laquelle (par exemple dans `accept`)

Sortie réelle

```
$ ./addr 7000
...
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28,
sockaddr_un 110, sockaddr_storage 128
```

Diapositive 187: il manquait htons

C

```
/* slide 187 */  
s.sin_port = 7000;           /* wrong */  
/* fixed (sock.c) */  
s.sin_port = htons(7000);    /* network order */  
s.sin_addr.s_addr = htonl(INADDR_ANY);
```

Sortie réelle

```
$ ./addr 7000  
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order  
sin_port = 7000:           the kernel bound port 22555  
sin_port = htons(7000):    the kernel bound port 7000  
...
```

- Le réseau utilise l'ordre *big-endian* ; x86-64 et ARM64 sont *little-endian* (vous l'avez vu dans le TD1 avec `od`)

0x1B58 lu à l'envers donne 0x581B = **22555** : le serveur écoute sur un autre port et personne ne le trouve.

De gethostbyname à getaddrinfo

C tds/td7/addr.c

```
const char *names[][2] = { { "localhost", "http" }, { "
for (int k = 0; k < 2; k++) {
    struct addrinfo hints = { .ai_family = AF_UNSPEC, .
    int e = getaddrinfo(names[k][0], names[k][1], &hints,
    exit_if(e != 0, gai_strerror(e));
    for (struct addrinfo *p = res; p != NULL; p = p->ai
...
    freeaddrinfo(res);
}
```

Sortie réelle

```
$ ./addr 7000
...
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
...
```

- **gethostbyname** (diapositive 187): IPv4 seulement, non sûr avec les threads, hors de POSIX depuis 2008
- **getaddrinfo** (RFC 3493): nom + service → liste d'adresses IPv4 et IPv6, prêtes pour **connect**

`int listen(int sock, int backlog);`

- Déclare le service au système: le socket devient **passif**, il ne fait qu'attendre des connexions
- `backlog` : combien de connexions **déjà établies** peuvent attendre `accept` (Linux en admet `backlog + 1`)
- Avec la file pleine, le SYN est jeté et le client le renvoie (1 s, 2 s, 4 s...)

Sortie réelle

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5, SYN_SENT 3
accept 5 (total 5), wait:  accept queue 3 | clients ESTABLISHED 8, SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```

accept : prendre une connexion dans la file

```
C
int accept(int sock, struct sockaddr *a, socklen_t *len);  /* a new descriptor */
```

- `a` et `*len` reçoivent l'adresse du client (ou `NULL` si on n'en a pas besoin)
- Bloque jusqu'à ce qu'il y ait une connexion dans la file
- Renvoie un **nouveau descripteur** pour ce client; celui d'écoute continue d'accepter

```
C tds/td7/sock.c
```

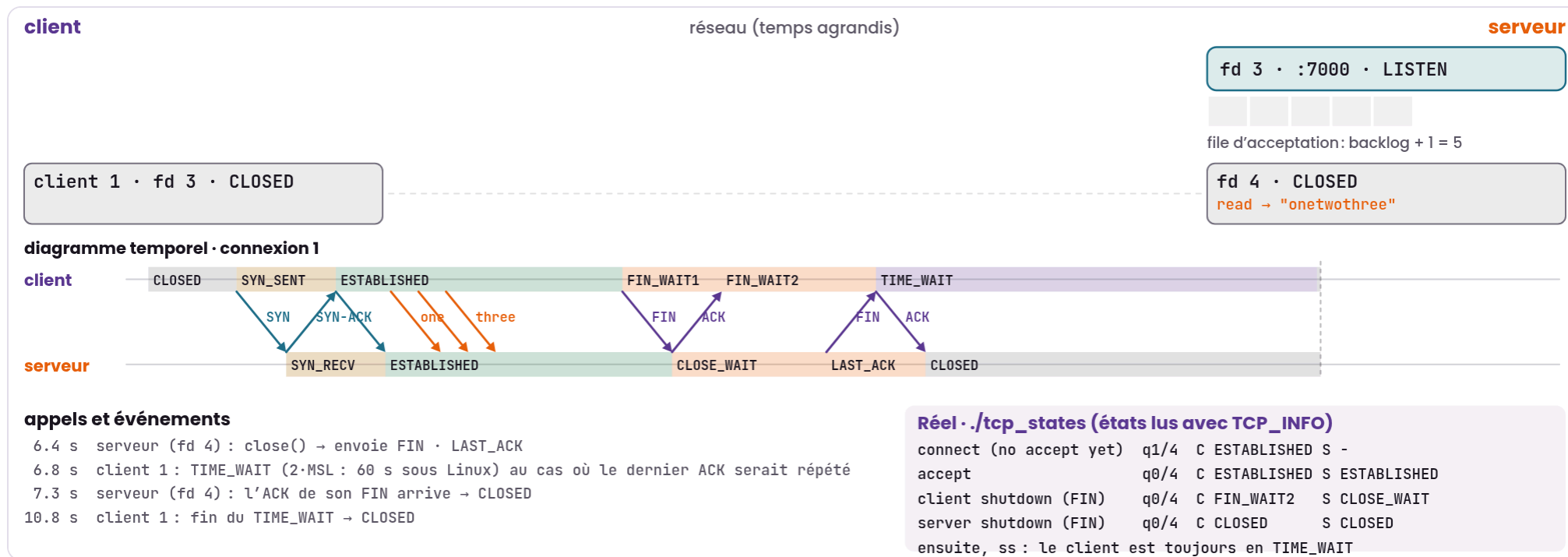
```
/* 5. accept blocks until a client connects and returns
int client = accept(s, NULL, NULL);
exit_if(client == -1, "accept");
printf("connection accepted, closing\n");
```

Correction (diapositive 189): il renvoie un descripteur ≥ 0 , et non « > 0 » (0 est valide si l'entrée standard a été fermée), ou -1 en cas d'erreur. `accept4(..., SOCK_CLOEXEC)` empêche le descripteur de fuir vers un `exec`.

connect : demander la connexion

- Demande la connexion à l'adresse du serveur: le noyau fait la **poignée de main en trois temps** (SYN, SYN-ACK, ACK)
- Bloque jusqu'à la fin de la poignée de main: **il n'attend pas accept**
- Erreurs typiques: `ECONNREFUSED` (personne n'écoute sur ce port), `ETIMEDOUT`
- Avec `SOCK_NONBLOCK`, il renvoie `EINPROGRESS` et la poignée de main continue toute seule (c'est ainsi que `backlog.c` ouvre 8 connexions à la fois)

Une connexion, segment par segment



Avec le descripteur renvoyé par `accept`

- Écrire: `write(fd, buf, n)` ou `send(fd, buf, n, flags)`
- Lire: `read(fd, buf, n)` ou `recv(fd, buf, n, flags)`
- Fermer: `close(fd)` ; `shutdown(fd, SHUT_WR)`
ne ferme que le sens écriture (envoie FIN)

Ce que vous savez des unités 2 et 5 s'applique ici:

- Lectures et écritures **partielles** : boucle (`write_all`)
- `read` → 0: l'autre extrémité a fermé
- Écrire sur un socket fermé → `SIGPIPE` ; on l'évite avec `MSG_NOSIGNAL` ou en ignorant le signal

Les états, lus dans le noyau avec TCP_INFO

Sortie réelle

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -          | server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED | server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED | server ESTABLISHED
write/read                  | server read 6 bytes
client shutdown (FIN)      | listener LISTEN (queue 0/4) | client FIN_WAIT2   | server CLOSE_WAIT
server read                 | server read returns 0: end of file
server shutdown (FIN)      | listener LISTEN (queue 0/4) | client CLOSED      | server CLOSED
                           | the client's end moved to a TIME_WAIT entry of the kernel (60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0        0        127.0.0.1:42772    127.0.0.1:PORT
```

Mode non connecté: `sendto` et `recvfrom`

C

```
ssize_t sendto(int sock, const void *buf, size_t len, int flags,  
               const struct sockaddr *dest, socklen_t dlen);  
ssize_t recvfrom(int sock, void *buf, size_t len, int flags,  
                 struct sockaddr *src, socklen_t *slen);
```

- `sendto` = `connect` + `send` en un seul appel; `recvfrom` = `accept` + `recv`, et indique qui a envoyé
- Ils renvoient les octets envoyés ou reçus (-1 en cas d'erreur); un `recvfrom` = **un datagramme entier** (si le tampon est trop court, le reste est perdu)
- On peut aussi faire `connect` sur un socket UDP: cela fixe la destination et permet `send` / `recv`

Qui appelle quoi, dans l'ordre

CONNECTÉ (TCP)	
SERVEUR	CLIENT
socket · bind · listen	socket
accept (bloque)	connect → poignée de main
read ⇔ write	write ⇔ read
close	close

NON CONNECTÉ (UDP)	
SERVEUR	CLIENT
socket · bind	socket
recvfrom (bloque)	sendto
sendto	recvfrom
close	close

Seul le serveur lie un port connu; le client reçoit automatiquement un port éphémère.

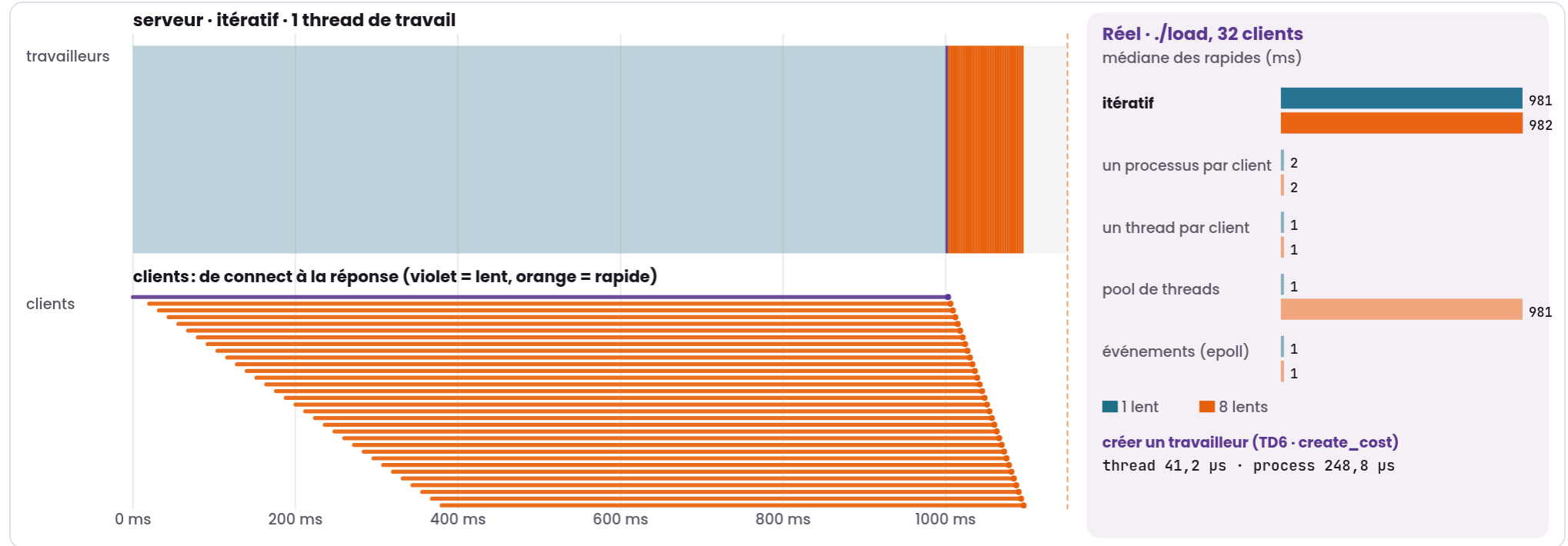
Modèles de serveur

- **Itératif** : un client à la fois
- **Multiprocessus ou multithread** : le thread principal fait `accept`, crée un processus ou un thread pour le service et revient à `accept`
- **Pool de threads** : la même chose, avec un nombre limité de threads. Producteur: le thread principal (connexions); consommateurs: les threads de service

Aujourd'hui aussi

Boucle d'événements : un thread et `epoll` (Linux, 2002) servent des milliers de connexions; c'est la réponse au problème C10K (D. Kegel, 1999) et le modèle de nginx ou de Node.js. `io_uring` (Linux 5.1, 2019) pousse l'idée plus loin: des files partagées avec le noyau.

Qui attend qui?



32 clients; d'abord 1, puis 8 lents (1 s)

Sortie réelle

```
$ ./http_server -m iterative & ./load PORT 1 31 1000;  
./load PORT 8 24 1000  
32 replies 200 OK of 32  
slow 1: median 1001 ms, max 1001 ms  
fast 31: median 981 ms, max 982 ms  
32 replies 200 OK of 32  
slow 8: median 1001 ms, max 1001 ms  
fast 24: median 982 ms, max 983 ms
```

Sortie réelle

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load  
PORT 8 24 1000  
32 replies 200 OK of 32  
slow 1: median 1001 ms, max 1001 ms  
fast 31: median 1 ms, max 2 ms  
32 replies 200 OK of 32  
slow 8: median 1001 ms, max 1001 ms  
fast 24: median 1 ms, max 2 ms
```

Le serveur itératif fait attendre tout le monde pour un seul client lent; `epoll`, avec un seul thread, non. Le pool de 8 tient avec 1 client lent mais pas avec 8 (A8.2).

Conclusion

- Un socket est un descripteur: `read`, `write` et `close` comme avec un fichier ou un tube
- TCP: flux fiable, connexion établie par le noyau; UDP: datagrammes sans garanties
- Adresses dans l'ordre réseau (`htons`) et résolues avec `getaddrinfo`
- Le modèle de serveur décide qui attend: processus, threads, pool ou événements

TD7 corrigé + défi IA

- `sock`, `echo_server` (itératif et avec `fork`)
- `http_server` avec cinq modèles
- Nouveau: `tcp_states`, `backlog`, `load`, `udp`, `addr`, `local`

Bash

```
make -C tds && tds/tests/run.sh td7
```



Universidad
Indoamérica

À RETENIR

Trois questions

1. Pourquoi `connect` peut-il se terminer avant que le serveur appelle `accept` ?
2. Quel port `sin_port = 7000` ouvre-t-il sur votre portable, et pourquoi ?
3. Quel modèle de serveur tient avec 8 clients lents en utilisant un seul thread ?